

# AI-POWERED PHISHING DETECTION SYSTEM

## Software Requirements Specification (SRS)

|                     |                                         |
|---------------------|-----------------------------------------|
| Document Type       | Software Requirements Specification     |
| Project Domain      | Artificial Intelligence / Cybersecurity |
| Primary Application | Phishing URL Detection                  |
| Detection Approach  | Machine Learning                        |
| Document Version    | 1.0                                     |
| Status              | Requirements Baseline                   |

**Purpose:** This document defines the functional, non-functional, technical, security, data, machine-learning, evaluation, and deployment requirements for the AI-Powered Phishing Detection System.

# 1. Introduction

Phishing is a cybersecurity threat in which attackers attempt to deceive users into interacting with malicious websites, URLs, messages, or other resources in order to obtain sensitive information. Phishing websites frequently imitate trusted services and may be distributed through email, messaging platforms, advertisements, search results, and social networks.

The proposed system focuses on automated phishing detection using URL-based machine-learning techniques. Research in IEEE Access has demonstrated the use of lexical and structural URL characteristics with machine-learning models, including ensemble methods, for phishing classification. Recent research has also emphasized early detection, explainability, and cross-dataset validation.

The system is intended to provide a practical detection platform while also supporting reproducible experimentation and comparative evaluation of machine-learning models.

## 2. Purpose

The purpose of the system is to determine whether a submitted URL is likely to represent a legitimate resource or a phishing resource using machine-learning-based analysis.

The system shall transform a URL into a structured representation of features, apply a trained classification model, and return a prediction with an associated risk score.

The system shall also provide an experimental framework for comparing machine-learning algorithms and evaluating their ability to generalize to previously unseen data.

## 3. System Scope

### 3.1 In Scope

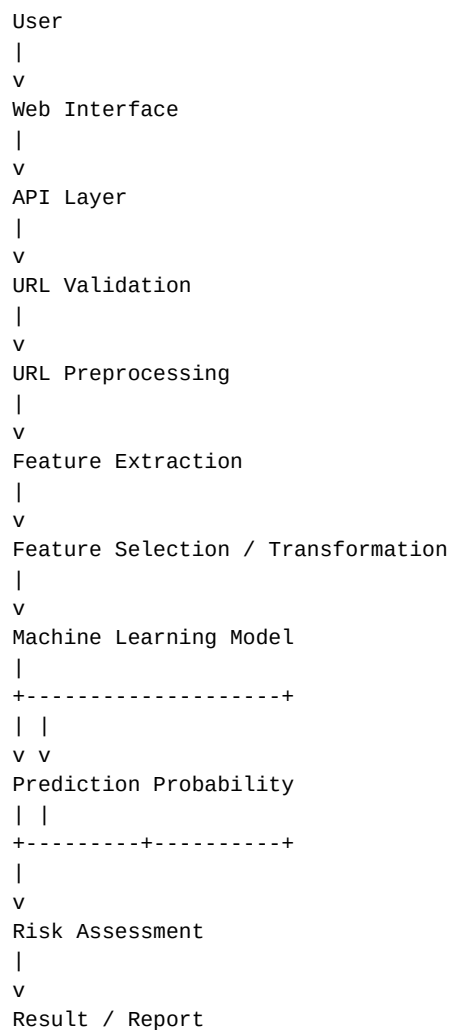
- URL submission and validation
- URL preprocessing and normalization
- Lexical URL feature extraction
- Structural URL feature extraction
- Domain-related feature extraction where available
- Feature preprocessing and selection
- Machine-learning model training
- Machine-learning model comparison
- Phishing probability estimation
- Risk-level classification
- Prediction explanation
- Model evaluation
- Cross-dataset validation
- Web-based detection interface

- REST API
- Prediction logging
- Performance monitoring

### 3.2 Out of Scope

- Automatic credential collection
- Automatic execution of malicious URLs
- Automated exploitation of websites
- Malware execution
- Unauthorized penetration testing
- Guaranteed detection of every phishing attack
- Full browser/network interception in the initial release
- Automatic modification of third-party websites

## 4. System Overview



The architecture separates URL processing, feature engineering, machine learning, prediction, and presentation so that individual components can be modified or evaluated independently.

## 5. Functional Requirements

### **FR-001 — URL Submission**

The system shall provide an interface through which a user or client application can submit a URL for analysis.

### **FR-002 — URL Validation**

The system shall validate submitted URLs before analysis and shall reject empty, malformed, unsupported, or invalid URL inputs.

### **FR-003 — URL Normalization**

The system shall normalize URLs into a consistent representation before feature extraction.

### **FR-004 — Feature Extraction**

The system shall extract measurable characteristics from the URL, including lexical, structural, and domain-related characteristics where available.

### **FR-005 — Feature Processing**

The system shall preprocess extracted features using the same transformation pipeline used during model training.

### **FR-006 — Feature Selection**

The system shall support feature selection or dimensionality reduction when required by the selected experimental methodology.

### **FR-007 — Model Loading**

The production inference service shall load a validated and versioned machine-learning model.

### **FR-008 — URL Classification**

The system shall classify a submitted URL as legitimate or phishing. A configurable intermediate suspicious category may be supported.

### **FR-009 — Probability Estimation**

The system shall return a probability or confidence score associated with the phishing classification.

### **FR-010 — Risk Classification**

The system shall map the model output to a configurable risk category.

### **FR-011 — Prediction Explanation**

The system shall expose important features or explanatory information associated with the prediction where supported by the selected model.

### **FR-012 — Prediction Record**

The system shall optionally record the analysis identifier, timestamp, model version, prediction, and relevant evaluation metadata.

### **FR-013 — Report Generation**

The system shall support generation of an analysis report containing the URL analysis result and associated model information.

### **FR-014 — Error Handling**

The system shall return meaningful error responses for invalid input, missing features, unavailable models, and internal processing failures.

## 6. URL Feature Requirements

The feature extraction component shall support a configurable feature set. The final feature set shall be documented and versioned with the model.

| Category    | Example Features                                   |
|-------------|----------------------------------------------------|
| URL lexical | URL length, path length, query length, digit count |
| Character   | Dots, hyphens, slashes, special characters         |
| Domain      | Domain length, subdomain count, TLD information    |
| Structure   | Path depth, query parameters, fragment presence    |
| Security    | HTTPS usage, IP-address usage                      |
| Textual     | Suspicious keywords, token patterns                |
| Statistical | Entropy and character-distribution features        |

Feature definitions shall be documented so that the training and inference pipelines produce identical feature representations.

## 7. Machine Learning Requirements

### ML-001 — Training Pipeline

The system shall provide a reproducible training pipeline covering data loading, preprocessing, feature preparation, model training, and evaluation.

### ML-002 — Baseline Models

The experimental framework shall support baseline models such as Logistic Regression, Decision Tree, Random Forest, SVM, or equivalent baseline classifiers.

### ML-003 — Boosting Models

The framework shall support gradient-boosting models, including XGBoost and/or LightGBM, for comparative evaluation.

### ML-004 — Hyperparameter Optimization

The framework shall support configurable hyperparameter optimization using an appropriate search strategy.

### ML-005 — Model Versioning

Every production model shall have an identifiable version associated with its training configuration and feature schema.

### ML-006 — Reproducibility

Training experiments shall record dataset versions, feature versions, model parameters, evaluation configuration, and random seeds where applicable.

## 8. Dataset Requirements

### DATA-001 — Dataset Documentation

Every dataset shall have documented source, collection information, class definitions, feature definitions, licensing or usage conditions, and preprocessing procedures.

### DATA-002 — Class Labels

The dataset shall distinguish phishing and legitimate samples using well-defined labels.

### DATA-003 — Data Quality

The preprocessing pipeline shall identify duplicate records, missing values, malformed URLs, inconsistent labels, and other relevant data-quality issues.

**DATA-004 — Class Distribution**

The class distribution shall be measured and reported. Appropriate balancing or weighting techniques shall be applied where justified.

**DATA-005 — Data Leakage Prevention**

Training, validation, and test data shall be separated in a manner that minimizes duplicate or related samples leaking between partitions.

## 9. Cross-Dataset Validation Requirements

Cross-dataset validation is a core research requirement. The purpose is to determine whether the detection model generalizes beyond the dataset used for model development.

**CV-001 — Independent Dataset Testing**

The system shall support evaluation on at least one dataset that is independent of the primary training dataset.

**CV-002 — Dataset-Specific Results**

Performance results shall be reported separately for each evaluation dataset.

**CV-003 — Generalization Analysis**

The system shall compare within-dataset and cross-dataset performance and identify significant performance changes.

**CV-004 — Feature Stability**

The analysis shall investigate whether the selected features remain effective across different datasets.

This requirement is motivated by recent IEEE Access research showing that strong performance on a custom dataset does not necessarily translate to equivalent performance on independent datasets. The 2026 XGBoost study reported 90.7% accuracy on its custom dataset and 77.8% average accuracy across three independent test sets.

## 10. Model Evaluation Requirements

| Metric              | Purpose                                          |
|---------------------|--------------------------------------------------|
| Accuracy            | Overall classification correctness               |
| Precision           | Reliability of positive/phishing predictions     |
| Recall              | Ability to identify phishing samples             |
| F1-score            | Balance between precision and recall             |
| Specificity         | Ability to correctly identify legitimate samples |
| False Positive Rate | Rate of legitimate URLs incorrectly classified   |
| False Negative Rate | Rate of phishing URLs missed by the model        |
| ROC-AUC             | Ranking/discrimination performance               |
| Inference Time      | Operational prediction latency                   |

**EVAL-001 — Confusion Matrix**

The system shall generate confusion matrices for applicable classification experiments.

**EVAL-002 — Model Comparison**

The experimental framework shall compare models using consistent datasets, features, partitions, and evaluation metrics.

### EVAL-003 — Error Analysis

The evaluation shall investigate representative false-positive and false-negative cases.

### EVAL-004 — Benchmarking

The final model selection shall consider detection performance, generalization, computational requirements, and deployment suitability.

## 11. Explainability Requirements

### XAI-001 — Feature Importance

The system shall expose feature-importance information when supported by the selected model.

### XAI-002 — Prediction Explanation

The system should provide human-readable information describing important characteristics associated with a prediction.

### XAI-003 — Explanation Consistency

Explanations shall correspond to the model version and feature schema used to produce the prediction.

## 12. Web Application Requirements

### WEB-001 — Detection Interface

The application shall provide a web interface for submitting a URL.

### WEB-002 — Result Display

The interface shall display prediction, probability/risk score, risk category, and relevant explanatory information.

### WEB-003 — API Integration

The web interface shall communicate with the backend through a defined API.

### WEB-004 — Responsive Interface

The interface shall remain usable on common desktop and mobile screen sizes.

### WEB-005 — Failure Feedback

The interface shall provide understandable feedback when analysis fails.

## 13. API Requirements

| Endpoint          | Method | Purpose                         |
|-------------------|--------|---------------------------------|
| /api/health       | GET    | Service health check            |
| /api/analyze      | POST   | Analyze submitted URL           |
| /api/models       | GET    | Return available model metadata |
| /api/reports/{id} | GET    | Retrieve analysis report        |

### API-001 — Input Validation

All API inputs shall be validated before processing.

### API-002 — Structured Responses

The API shall return consistent machine-readable response structures.

### API-003 — Error Codes

The API shall provide appropriate HTTP status codes and meaningful error messages.

## 14. Non-Functional Requirements

## 14.1 Performance

### NFR-001 — Inference Performance

The production inference pipeline should provide interactive response times under normal operating conditions. The actual measured latency shall be reported as an experimental result.

## 14.2 Reliability

### NFR-002 — Fault Tolerance

Invalid URLs, malformed input, missing features, model failures, and temporary service failures shall be handled without crashing the application.

## 14.3 Maintainability

### NFR-003 — Modular Architecture

Frontend, API, feature extraction, model inference, data access, and evaluation components shall be modular.

## 14.4 Scalability

### NFR-004 — Extensibility

The architecture shall permit additional datasets, feature sets, models, and detection methods to be added without redesigning the entire system.

## 14.5 Portability

### NFR-005 — Deployment Portability

The application should support deployment on standard Linux, macOS, or Windows development environments and suitable cloud infrastructure.

# 15. Security Requirements

### SEC-001 — Input Sanitization

User-provided data shall be validated and safely handled before being processed by application components.

### SEC-002 — No Automatic Navigation

The initial URL-analysis pipeline shall not automatically navigate to or execute content from a submitted suspicious URL.

### SEC-003 — API Protection

Production API endpoints shall implement appropriate authentication, rate limiting, input validation, and access controls where required.

### SEC-004 — Sensitive Configuration

Secrets, API keys, database credentials, and signing keys shall not be hard-coded in source code.

### SEC-005 — Logging

Security-relevant application errors and operational events shall be logged without unnecessarily storing sensitive user information.

# 16. Privacy Requirements

### PRIV-001 — Data Minimization

Only information required for detection, evaluation, auditing, or explicitly enabled reporting functionality shall be stored.

### PRIV-002 — Retention

Stored URLs and analysis records shall have a configurable retention policy.



### PRIV-003 — Access Control

Stored analysis data shall only be accessible to authorized components and users.

## 17. Hardware Requirements

| Component | Minimum             | Recommended      |
|-----------|---------------------|------------------|
| CPU       | 4 cores             | 6–8 cores        |
| RAM       | 8 GB                | 16 GB+           |
| Storage   | 20 GB               | 50 GB+           |
| GPU       | Not required        | Optional         |
| Network   | Internet connection | Stable broadband |

GPU acceleration is not a mandatory requirement for the primary URL-feature machine-learning pipeline. CPU-based training and inference are expected to be sufficient for conventional tabular ML models.

## 18. Software Requirements

| Category            | Recommended Technology |
|---------------------|------------------------|
| Language            | Python 3.x             |
| Data Processing     | Pandas, NumPy          |
| Machine Learning    | Scikit-learn           |
| Boosting            | XGBoost, LightGBM      |
| Visualization       | Matplotlib             |
| Backend             | FastAPI                |
| Frontend            | HTML, CSS, JavaScript  |
| Database            | MongoDB or PostgreSQL  |
| Model Serialization | Joblib                 |
| Version Control     | Git                    |

## 19. System Constraints

- Detection quality depends on the quality and representativeness of training data.
- The model cannot guarantee detection of previously unseen phishing strategies.
- Dataset imbalance may influence evaluation results.
- Cross-dataset performance may be lower than within-dataset performance.
- URL-only analysis cannot capture all webpage-level visual or behavioral indicators.
- External threat-intelligence services may impose rate limits or availability constraints.
- Model predictions should be treated as automated risk assessments rather than absolute security guarantees.

## 20. Future Extensions

- HTML and DOM feature analysis
- Webpage screenshot analysis
- Computer-vision-based phishing detection
- Multimodal URL + HTML + screenshot analysis
- Browser extension integration
- Real-time threat-intelligence integration
- Adversarial robustness testing
- Continuous model retraining
- Online learning
- Community-based phishing reporting

## 21. Acceptance Criteria

- A valid URL can be submitted through the application.
- Invalid input is rejected safely.
- The URL is converted into the required feature representation.
- A trained model produces a classification.
- A phishing probability or confidence score is returned.
- A configurable risk level is displayed.
- The model version is identifiable.
- Model performance is measured using multiple appropriate metrics.
- At least one independent dataset is available for cross-dataset validation.
- False-positive and false-negative cases are analyzed.
- The API returns structured responses.
- The web interface displays analysis results correctly.
- The system handles application errors without terminating unexpectedly.
- The final model and feature pipeline are reproducible from documented configuration.

## 22. Requirements Traceability Matrix

| Requirement Area   | Implementation Component | Validation              |
|--------------------|--------------------------|-------------------------|
| URL Input          | Frontend + API           | Input test cases        |
| URL Validation     | Validation module        | Valid/invalid URL tests |
| Feature Extraction | Feature pipeline         | Feature unit tests      |

| Requirement Area | Implementation Component | Validation                  |
|------------------|--------------------------|-----------------------------|
| Model Training   | Training pipeline        | Training experiment         |
| Model Comparison | Evaluation pipeline      | Benchmark results           |
| Prediction       | Inference service        | Prediction tests            |
| Explainability   | XAI module               | Feature importance          |
| Cross-Dataset    | Evaluation pipeline      | Independent test dataset    |
| Security         | API/security layer       | Security test cases         |
| Performance      | Inference service        | Latency benchmark           |
| Reporting        | Report module            | Generated report validation |

## 23. Research References

- Karim et al., “Phishing Detection System Through Hybrid Machine Learning Based on URL,” IEEE Access, Vol. 11, 2023, pp. 36805–36822. DOI: 10.1109/ACCESS.2023.3252366.
- PhishCatcher, “Client-Side Defense Against Web Spoofing Attacks Using Machine Learning,” IEEE Access, Vol. 11, 2023, pp. 61249–61263. DOI: 10.1109/ACCESS.2023.3287226.
- Duarte et al., “Machine Learning for Early Detection of Phishing URLs in Parked Domains: An Approach Applied to a Financial Institution,” IEEE Access, Vol. 13, 2025, pp. 145736–145753. DOI: 10.1109/ACCESS.2025.3599454.
- Misiek and Hyla, “XGBoost-Based URL Phishing Detection Method With Cross-Dataset Validation,” IEEE Access, Vol. 14, 2026, pp. 43200–43212. DOI: 10.1109/ACCESS.2026.3672690.